

Day 6 — HAVING + CASE

Today is an important day because you'll combine what you've learned so far:

SELECT

WHERE

ORDER BY

LIMIT

DISTINCT

COUNT / SUM / AVG / MIN / MAX

GROUP BY

with two powerful SQL concepts:

- HAVING → filter groups
- CASE → create conditional logic

These are heavily used in analytics, reporting, dashboards, data pipelines, and RAG monitoring.

Rule: You write the SQL first. I will not provide answers before you attempt them.

1. Today's Goals

By the end of Day 6, you should be able to:

- Explain WHERE vs HAVING
 - Filter aggregated groups with HAVING
 - Use HAVING with COUNT, SUM, AVG, MIN, MAX
 - Combine WHERE + GROUP BY + HAVING
 - Use CASE WHEN
 - Create categories from numeric values
 - Create labels from statuses
 - Use CASE with aggregates
 - Understand CASE evaluation
 - Debug common HAVING and CASE mistakes
 - Build realistic RAG monitoring queries
-

2. Our RAG Database

Continue with:

documents

id	user_id	title	file_type	page_count	status
101	1	RAG Fundamentals	pdf	20	processed

id	user_id	title	file_type	page_count	status
102	1	PostgreSQL Guide	pdf	45	processed
103	2	Python Basics	pdf	30	processing
104	3	FastAPI Tutorial	md	15	failed
105	4	Vector Databases	pdf	50	processed
106	5	Docker Guide	pdf	25	processing

PART 1 — HAVING

3. Why Do We Need HAVING?

Yesterday you learned:

```
SELECT
status,
COUNT(*) AS document_count
  FROM documents
 GROUP BY status;
```

This gives:

status	document_count
processed	3
processing	2
failed	1

Now imagine the requirement:

Show only statuses that have at least 2 documents.

You cannot use:

```
WHERE COUNT(*) >= 2
```

Why?

Because WHERE operates on individual rows **before grouping**.

You need:

HAVING

4. Basic HAVING

Conceptually:

```
SELECT
status,
COUNT(*) AS document_count
  FROM documents
 GROUP BY status
 HAVING COUNT(*) >= 2;
```

Result:

status	document_count
--------	----------------

processed	3
-----------	---

processing	2
------------	---

failed disappears because its count is only 1.

5. The Key Difference

Memorize this:

WHERE → filters rows

HAVING → filters groups

Example:

WHERE status = 'processed'

means:

Filter individual document rows.

While:

HAVING COUNT(*) >= 2

means:

After grouping, keep only groups with at least 2 rows.

6. Mental Model

This is extremely important.

Suppose:

SELECT

status,

COUNT(*) AS document_count

FROM documents

WHERE file_type = 'pdf'

GROUP BY status

HAVING COUNT(*) >= 2;

Conceptually:

ALL DOCUMENTS

↓

WHERE

↓

PDF documents only

↓

GROUP BY status

↓

Create status groups

↓

COUNT each group

↓

HAVING

↓

Keep groups with count ≥ 2

Think:

WHERE → before grouping

HAVING → after grouping

7. WHERE and HAVING Together

Example requirement:

Find file types having at least 2 processed documents.

You would conceptually:

1. Filter processed documents
2. Group by file type
3. Count each group
4. Keep groups with count ≥ 2

Structure:

SELECT

file_type,

COUNT(*) AS document_count

FROM documents

WHERE status = 'processed'

GROUP BY file_type

HAVING COUNT(*) ≥ 2 ;

This pattern is extremely common.

8. HAVING With SUM

Suppose:

Show users whose uploaded documents contain more than 50 total pages.

Conceptually:

```
SELECT
user_id,
SUM(page_count) AS total_pages
  FROM documents
  GROUP BY user_id
  HAVING SUM(page_count) > 50;
```

Notice:

SUM(page_count)

is calculated per user.

Then HAVING filters the user groups.

9. HAVING With AVG

Suppose:

Find users whose average document size is greater than 30 pages.

```
SELECT
user_id,
AVG(page_count) AS average_pages
  FROM documents
  GROUP BY user_id
  HAVING AVG(page_count) > 30;
```

Again:

GROUP

↓

AVG

↓

HAVING

10. HAVING With MAX

Suppose:

Find users who have uploaded at least one document larger than 40 pages.

You could conceptually use:

```
HAVING MAX(page_count) > 40
```

The aggregate is calculated for each user, then the group is filtered.

11. HAVING vs WHERE — Critical Example

Consider this requirement:

Find users who have at least 2 processed documents.

Correct conceptual flow:

processed documents

↓

group by user

↓

count

↓

keep count ≥ 2

So you need:

SELECT

user_id,

COUNT(*) AS processed_documents

FROM documents

WHERE status = 'processed'

GROUP BY user_id

HAVING COUNT(*) ≥ 2 ;

Notice the two filters do different jobs:

WHERE status = processed

filters **rows**.

HAVING COUNT(*) ≥ 2

filters **groups**.

PART 2 — CASE

12. What Is CASE?

CASE allows SQL to perform conditional logic.

Think of it as SQL's version of:

if

else if

else

Basic structure:

CASE

WHEN condition THEN result

ELSE result

END

13. Simple CASE Example

Suppose we want to classify documents based on size.

Business rule:

< 20 pages → Small

20–40 pages → Medium

> 40 pages → Large

You can write:

```
SELECT
title,
page_count,
CASE
WHEN page_count < 20 THEN 'Small'
WHEN page_count <= 40 THEN 'Medium'
ELSE 'Large'
END AS document_size
FROM documents;
```

The result might look like:

title	page_count	document_size
RAG Fundamentals	20	Medium
PostgreSQL Guide	45	Large
Python Basics	30	Medium
FastAPI Tutorial	15	Small
Vector Databases	50	Large
Docker Guide	25	Medium

14. CASE Evaluates Conditions in Order

This is important.

Consider:

```
CASE
WHEN page_count > 40 THEN 'Large'
WHEN page_count > 20 THEN 'Medium'
ELSE 'Small'
END
For:
```

page_count = 50

First condition:

50 > 40

TRUE.

So PostgreSQL returns:

Large

It doesn't continue looking for another matching condition.

15. Order Matters in CASE

Compare:

CASE

WHEN page_count > 20 THEN 'Medium'

WHEN page_count > 40 THEN 'Large'

ELSE 'Small'

END

For 50 pages:

50 > 20

is already true.

So it gets:

Medium

even though 50 is also greater than 40.

Rule

CASE stops at the first matching WHEN.

This is a common source of bugs.

16. CASE for Status Labels

Suppose your database has:

processed

processing

failed

But your frontend wants human-readable labels.

You can create:

SELECT

title,

status,

```
CASE
WHEN status = 'processed' THEN 'Completed'
WHEN status = 'processing' THEN 'In Progress'
WHEN status = 'failed' THEN 'Needs Attention'
ELSE 'Unknown'
END AS status_label
FROM documents;
```

The original status isn't changed.

You're creating a calculated output column.

17. CASE Does Not Modify Data

This:

```
CASE
WHEN status = 'failed' THEN 'Needs Attention'
END
```

does not update the table.

It's only producing a value in the query result.

This is similar to AS:

Query result transformation, not schema modification.

18. CASE With Numbers

You can create risk categories.

For example:

page_count < 20 → low

20–40 → medium

> 40 → high

SQL:

```
SELECT
title,
page_count,
CASE
WHEN page_count < 20 THEN 'low'
WHEN page_count <= 40 THEN 'medium'
ELSE 'high'
END AS processing_risk
```

FROM documents;

This type of classification is common in analytics.

19. CASE With NULL

Suppose page_count can be NULL.

You can explicitly handle it:

CASE

WHEN page_count IS NULL THEN 'Unknown'

WHEN page_count < 20 THEN 'Small'

WHEN page_count <= 40 THEN 'Medium'

ELSE 'Large'

END AS document_size

Notice:

IS NULL

not:

= NULL

20. CASE + Aggregate

Now things become more interesting.

You can use CASE inside an aggregate.

Suppose:

Count failed documents.

One approach we've already learned:

```
SELECT COUNT(*)
```

```
FROM documents
```

```
WHERE status = 'failed';
```

But conditional aggregation allows patterns like:

```
SELECT
```

```
COUNT(
```

```
CASE
```

```
WHEN status = 'failed' THEN 1
```

```
END
```

```
) AS failed_documents
```

```
FROM documents;
```

Don't worry if this feels strange.

The idea is:

failed row

↓

CASE → 1

other row

↓

CASE → NULL

COUNT ignores NULL

So only failed rows are counted.

We'll use this technique heavily later.

21. Multiple Conditional Metrics

You can calculate several metrics in one query:

```
SELECT
COUNT(*) AS total_documents,
COUNT(
CASE
WHEN status = 'processed' THEN 1
END
) AS processed_documents,
COUNT(
CASE
WHEN status = 'processing' THEN 1
END
) AS processing_documents,
COUNT(
CASE
WHEN status = 'failed' THEN 1
END
) AS failed_documents
FROM documents;
```

This produces one dashboard row.

This is a valuable pattern for real-world SQL.

22. Another Conditional Aggregation Pattern

You can use SUM():

```
SELECT
SUM(
CASE
WHEN status = 'failed' THEN 1
ELSE 0
END
) AS failed_documents
FROM documents;
```

Think:

failed → 1

processed → 0

processing → 0

failed → 1

Then:

$1 + 0 + 0 + 1 = 2$

The exact data determines the result.

23. CASE for Business Logic

Imagine your RAG system has a processing SLA:

processed → Healthy

processing → In Progress

failed → Alert

You can expose this directly to an application:

```
SELECT
id,
title,
CASE
WHEN status = 'processed' THEN 'Healthy'
WHEN status = 'processing' THEN 'In Progress'
WHEN status = 'failed' THEN 'Alert'
ELSE 'Unknown'
END AS pipeline_state
FROM documents;
```

This is often useful when SQL acts as a transformation layer between a database and API.

24. CASE vs WHERE

These are different.

WHERE

Removes rows:

```
WHERE status = 'failed'
```

Result contains only failed rows.

CASE

Keeps rows but changes/calculates a value:

```
CASE
```

```
WHEN status = 'failed' THEN 'Alert'
```

```
ELSE 'Normal'
```

```
END
```

All rows can still be returned.

Remember:

```
WHERE → filter rows
```

```
CASE → calculate/transform values
```

25. CASE + GROUP BY

You can also group based on a calculated category.

For example, classify document size and count each category:

```
SELECT
```

```
CASE
```

```
WHEN page_count < 20 THEN 'Small'
```

```
WHEN page_count <= 40 THEN 'Medium'
```

```
ELSE 'Large'
```

```
END AS document_size,
```

```
COUNT(*) AS document_count
```

```
FROM documents
```

```
GROUP BY
```

```
CASE
```

```
WHEN page_count < 20 THEN 'Small'
```

```
WHEN page_count <= 40 THEN 'Medium'
```

```
ELSE 'Large'
```

END;

Don't worry about making this elegant yet.

The important concept:

You can group rows based on a calculated category.

26. Common HAVING Mistake

Wrong:

```
SELECT
```

```
status,
```

```
COUNT(*) AS document_count
```

```
FROM documents
```

```
GROUP BY status
```

```
WHERE COUNT(*) > 1;
```

Problems:

- WHERE is in the wrong position.
 - Aggregate filtering belongs in HAVING.
-

27. Common HAVING Mistake

Wrong:

```
SELECT
```

```
status,
```

```
COUNT(*) AS document_count
```

```
FROM documents
```

```
GROUP BY status
```

```
HAVING document_count > 1;
```

Depending on SQL context, using a select-list alias in HAVING is not the safe pattern to learn here.

Prefer:

```
HAVING COUNT(*) > 1
```

We'll stick to explicit aggregate expressions.

28. Common CASE Mistake

Wrong:

```
CASE
```

```
WHEN page_count > 20 THEN 'Medium'
```

```
WHEN page_count > 40 THEN 'Large'  
ELSE 'Small'  
END
```

Why?

A 50-page document satisfies:

first.

So it gets "Medium".

Put more specific conditions first:

> 40

> 20

else

29. Debugging Challenge #1 — HAVING

Developer writes:

```
SELECT  
user_id,  
COUNT(*) AS document_count  
FROM documents  
GROUP BY user_id  
WHERE COUNT(*) >= 2;
```

Requirement:

Show users who have at least two documents.

Your tasks:

1. Identify the errors.
 2. Explain why WHERE COUNT(*) is wrong.
 3. Rewrite the query.
 4. Explain the execution logic.
-

30. Debugging Challenge #2 — CASE

Developer writes:

```
SELECT  
title,  
page_count,  
CASE  
WHEN page_count > 20 THEN 'Medium'
```

```
WHEN page_count > 40 THEN 'Large'  
ELSE 'Small'  
END AS size  
FROM documents;
```

Your tasks:

1. What's logically wrong?
 2. What happens to a 50-page document?
 3. Rewrite the CASE correctly.
-

31. Debugging Challenge #3

Developer wants:

Count users who have more than 50 total document pages.

They write:

```
SELECT  
user_id,  
SUM(page_count) AS total_pages  
FROM documents  
WHERE SUM(page_count) > 50  
GROUP BY user_id;
```

Your tasks:

1. What's wrong?
 2. Should SUM(page_count) > 50 be in WHERE or HAVING?
 3. Rewrite it.
-

32. Exercise Set A — HAVING

Q1

Count documents by status and show only statuses having at least 2 documents.

Q2

Group documents by file_type and show only file types with more than 2 documents.

Q3

Group documents by user_id and show users who have uploaded at least 2 documents.

Q4

Group documents by user_id and show users whose total pages exceed 40.

Q5

Group documents by user_id and show users whose average document size is greater than 25 pages.

33. Exercise Set B — WHERE + GROUP BY + HAVING

Q6

Find users who have at least 2 **processed** documents.

Q7

Find users whose processed documents contain more than 50 total pages.

Q8

Find file types having at least 2 documents with more than 20 pages.

Be careful:

WHERE

↓

GROUP BY

↓

HAVING

34. Exercise Set C — CASE

Q9

Return:

title

page_count

document_size

Classify:

< 20 → Small

20-40 → Medium

> 40 → Large

Q10

Return:

title

status

status_label

Classification:

processed → Completed

processing → In Progress

failed → Failed

Q11

Create:

title

page_count

processing_risk

Rules:

< 20 pages → Low

20-40 → Medium

> 40 → High

Q12

Handle NULL page counts:

NULL → Unknown

< 20 → Small

20-40 → Medium

> 40 → Large

35. Exercise Set D — Conditional Aggregation

Q13

Return one row containing:

total_documents

processed_documents

processing_documents

failed_documents

Use CASE + aggregate functions.

Q14

Return one row containing:

total_pages

processed_pages

processing_pages

failed_pages

Use conditional aggregation.

36. Practical Engineering Problem #1

You're building a RAG ingestion monitoring API.

The API needs:

status

document_count

total_pages

But it should **only show statuses having at least 2 documents**.

Requirements:

- Group by status
- Count documents
- Sum pages
- Filter groups using HAVING
- Sort by document count descending

Write one query.

37. Practical Engineering Problem #2

Your RAG application wants to classify every document for a processing dashboard.

Return:

document_id

title

page_count

size_category

pipeline_state

Rules:

Size

< 20 → Small

20–40 → Medium

> 40 → Large

Pipeline state

processed → Healthy

processing → Running

failed → Attention Required

Write one SQL query using **two CASE expressions**.

38. Practical Engineering Problem #3

Your RAG team wants to identify heavy users.

Definition:

A heavy user has at least 2 documents AND more than 50 total pages.

Return:

user_id

document_count

total_pages

Requirements:

- GROUP BY
 - COUNT
 - SUM
 - HAVING
 - One query
-

39. Practical Engineering Problem #4

Create a single dashboard row containing:

total_documents

processed_documents

processing_documents

failed_documents

large_documents

Definition of a large document:

page_count > 40

Use conditional aggregation.

Challenge: Do this without multiple separate queries.

40. Production Debugging

A developer wrote:

```
SELECT
user_id,
COUNT(*) AS document_count,
SUM(page_count) AS total_pages
  FROM documents
 WHERE status = 'processed'
 GROUP BY user_id
 HAVING document_count >= 2
```

AND total_pages > 50;

Requirement:

Return users having at least 2 processed documents and more than 50 processed pages.

Your tasks:

1. Identify the problem.
 2. Which parts are filtering rows?
 3. Which parts are filtering groups?
 4. Rewrite the query using the safe syntax we've learned.
 5. Explain your reasoning.
-

41. Interview Questions

Answer in your own words.

Q1

What's the difference between WHERE and HAVING?

Q2

Can WHERE directly filter the result of COUNT(*)?

Why or why not?

Q3

What does CASE do in SQL?

Q4

When SQL evaluates:

CASE

WHEN condition1 THEN result1

WHEN condition2 THEN result2

ELSE result3

END

and both conditions are true, which result is returned?

Q5

What is the difference between using CASE and WHERE?

Q6

Why can this be logically wrong?

CASE

WHEN page_count > 20 THEN 'Medium'

WHEN page_count > 40 THEN 'Large'

END

42. Explanation Challenge

Explain this query as if you're explaining it during a code review:

```
SELECT
```

```
  user_id,
```

```
  COUNT(*) AS document_count,
```

```
  SUM(page_count) AS total_pages,
```

```
  CASE
```

```
    WHEN SUM(page_count) > 100 THEN 'Heavy'
```

```
    WHEN SUM(page_count) > 50 THEN 'Medium'
```

```
    ELSE 'Light'
```

```
  END AS usage_level
```

```
  FROM documents
```

```
GROUP BY user_id
HAVING COUNT(*) >= 2
ORDER BY total_pages DESC;
```

Your explanation must answer:

1. What is one output row?
 2. How are documents grouped?
 3. What does COUNT(*) calculate?
 4. What does SUM(page_count) calculate?
 5. What does HAVING remove?
 6. What does CASE calculate?
 7. Why is the order of the CASE conditions important?
 8. How are the final rows sorted?
-

43. Data Engineering Thinking

Imagine your RAG database has:

10 million documents

You need a dashboard showing:

processed

processing

failed

with counts.

A developer proposes running three separate queries:

Query 1 → processed count

Query 2 → processing count

Query 3 → failed count

Another developer proposes one query using conditional aggregation.

Question

Why might the second approach be attractive?

Think about:

- Database round trips
- API latency
- Maintaining SQL
- Consistency of the metrics
- Query complexity

Don't assume "one query is always faster." Explain the trade-off.

44. Day 6 Short Test

Q1

What is the difference between WHERE and HAVING?

Q2

Write a query that returns statuses having at least 2 documents.

Q3

Where should this condition go?

COUNT(*) > 5

WHERE or HAVING?

Explain.

Q4

Write a CASE that classifies:

< 20 → Small

20–40 → Medium

> 40 → Large

Q5

What happens when multiple WHEN conditions are true?

Q6

What's wrong with:

WHERE COUNT(*) > 2

Q7

Write a query to find users having more than 50 total pages.

Q8

Write a query that labels:

processed → Healthy
processing → Running
failed → Alert

Q9

Write a query returning:

status

document_count

only for statuses with at least 2 documents.

Q10

Write one query returning:

total_documents

processed_documents

failed_documents

using conditional aggregation.
